



Information Retrieval Overview

A concise synthesis of four PDF presentations that together introduce IR fundamentals, index construction and compression, and practical PyLucene usage—including handling non-English text.



Content Sources

- Section 1: covers [search engine architecture](#), [indexing](#), and [PyLucene](#) basics for building a simple IR system from a 313-page PDF presentation. ([source](#))
- Section 2: explains [inverted index](#), [compression](#), and [Zipf's law](#) in large-scale IR systems from a 118-page PDF presentation. ([source](#))
- Section 3: introduces [PyLucene](#), [indexing](#), and [retrieval models](#) for hands-on IR practice from a 78-page PDF presentation. ([source](#))
- Section 4: provides a tutorial on [non-English text analysis](#), [PyLucene](#), and core [IR fundamentals](#) from a 40-page PDF presentation. ([source](#))

Section 1: Introduction to Information Retrieval (IR) – Course Overview and Foundations

This section covers the foundational concepts, course structure, and core principles of Information Retrieval from a 313-page NPTEL presentation by Dwaipayan Roy, IISER Kolkata ([source](#))



Course Goals and Learning Outcomes

Goal	Topics Covered
Understand how search engines work	• Architecture of search engines • Indexing and retrieval models
Build a simple search engine	• PyLucene implementation • Document representation & query processing
Study user behaviour	• Interaction and feedback loops
Analyse failures & evaluation	• Why search engines fail • Evaluation metrics (Precision, Recall, MAP, nDCG, etc.)
Hands-on with Lucene	• Indexing, querying, and analysis using Lucene/PyLucene



Why Information Retrieval Matters

Information Retrieval (IR) is the process of seeking out unstructured textual documents that satisfy a user's information need from within a document collection.

Key Statistics:

- Approximately **95% of the Web is text**
- Content includes: books, social-media posts, scientific papers, emails, etc.
- **Rapid growth** necessitates smarter search and analysis tools

IR Provides:

- Techniques to **organise** and **analyse** unstructured text

- Foundations for [search, discovery, learning, and recommendation](#)

Core Learning Outcomes

Skill	Example Application
Text handling	Morphology, synonym handling, determining topical similarity
Text analytics	Detect what makes a document set distinct
Search-engine building	Rank documents by relevance
Algorithm evaluation	Compare retrieval strategies (A vs. B) statistically
Interactive feedback	Use user feedback to improve ranking

Key References:

- Manning, Raghavan & Schütze – *Introduction to Information Retrieval* (Stanford)
- Butcher, Clarke & Cormack – *Information Retrieval: Implementing and Evaluating Search Engines* (Waterloo)

Foundations of IR

Definition of Key Terms

Information – a collection of facts from which conclusions can be drawn

Retrieval – the process of seeking something back from somewhere

Information Retrieval (IR) – The process of seeking out unstructured textual documents that satisfy a user's information need from within a document collection

Characteristics:

- **Unstructured nature** – documents are essentially bags of words, with no rigid schema
- **Information need** – expressed as a query (keywords or natural-language phrase)

Document Types

Type	IR Application
Text	Primary focus of this course
Audio	Speech-to-text, music retrieval
Video	Content-based retrieval, transcript search
Images	Image captioning, visual search

IR vs. Traditional DBMS

Aspect	IR (Documents)	DBMS (Records)
Query language	Natural-language / free-text	Structured (SQL)
Query completeness	Often incomplete, vague	Typically complete
Matching	Vague, relevance-based	Exact, Boolean
Result set	Ranked by relevance	All matching rows
Ranking	Core feature	Not applicable

Practical Applications

- Web search, desktop search, library search
- Product search, people search, question answering
- Digital libraries, enterprise search, federated search
- News, social, local-business, expert search
- Image, video, patent, micro-blog search
- **Recommender systems**

Week-wise Course Plan

Weeks	Topics
1-4	Intro, tokenisation, term filtering, term-document incidence matrix, Boolean retrieval, inverted index, skip pointers, PyLucene indexing, Luke viewer
5-8	Ranked retrieval (TF-IDF, cosine similarity, VSM), probabilistic models (BM25 family), language models (Jelinek-Mercer, Dirichlet), smoothing, Kullback-Leibler & Jensen-Shannon divergence, PyLucene query classes
9-12	Evaluation metrics (precision/recall, MAP, nDCG, MRR, kappa), TREC benchmarking, relevance feedback (Rocchio, RLM), crawling & PageRank/HITS/SEO, learning-to-rank, LSI, embeddings (word/sentence/document), BERT & LLMs in IR

Text Processing Pipeline

Document Representation – Two-step Process

1. **Text Processing (term selection & filtering)**
2. **Indexing (storing the terms)**

Term Selection Approaches

Approach	Description	Pros	Cons
Controlled Vocabulary	Pre-defined, expert-crafted set of topics	High precision	Low coverage, expensive
Free-Text Vocabulary	Uses (almost) all words from the document	Cheap, flexible, any detail level	May be ambiguous

Course adopts free-text vocabulary as the primary approach

Term Filtering Steps

Step	Description	Example/Issues
Tokenisation	Split raw text into atomic tokens	Ph.D., post-covid, O'Connor, non-English characters
Down-casing (case-folding)	Normalise case, often lower-casing	Keep all-caps titles, proper nouns at sentence start
Stop-word Removal	Eliminate very common words	am, is, the, at, for, with – list size varies (200-300 vs. 10-20)

Linguistic Processing

Technique	Goal	Example
Lemmatization	Reduce inflectional forms to dictionary head (lemma)	am, are, is → be; cars → car
Stemming	Heuristic truncation of suffixes	institution, institutional → institut

Porter Stemmer – widely used rule-based stemmer with 5 reduction phases

Key Insight: Stemming is fast but language-independent; lemmatization is more accurate but requires linguistic resources

Bag-of-Words (BoW) Model

- **Document → list of keywords** (after tokenisation, down-casing, stop-word removal, stemming/lemmatization)
- **Query → list of keywords** (same processing)
- **Retrieval** – measure overlap (e.g., term frequency, TF-IDF, cosine similarity) between query and document vectors



Inverted Index – The Core Data Structure

Forward vs. Inverted Index

Index Type	Mapping
Forward Index	Document → terms (e.g., Doc 1: Bruce loves programming)
Inverted Index	Term → list of document IDs (e.g., programming → [1, 3, 4])

Building an Inverted Index

1. **Token Sequencing** – generate (term, docID) pairs for each document
2. **Sorting** – first lexicographically by term, then by docID
3. **Merging** – combine duplicate term-docID pairs, compute term frequencies (TF) and document frequencies (DF)
4. **Storing** – split into two files:
 - **Dictionary** – sorted list of terms, each with DF, collection frequency (CF), and pointer to postings list
 - **Postings** – ordered list of docIDs (and optionally TF, positions)

Example Dictionary Entry

Term	DF	CF	Posting List (docIDs)
Alfred	8	1	2, 4, 7, 8, 16, 37, 54, 89
Arthur	11	1	1, 2, 5, 9, 11, 25, 31, 53, 68, 91, 94
Batman	4	1	2, 8, 37, 54
Bruce	7	1	2, 4, 8, 11, 18, 37, 54

Query Processing (Boolean Retrieval)

- **AND query** (Alfred AND Bruce):
 1. Retrieve postings for each term from dictionary
 2. **Merge** postings by intersecting sorted lists (linear scan, $O(|p_1| + |p_2|)$)
 3. Result: docIDs present in **both** lists
- **Optimization Tips**
 - Store **document frequency (df)** with each term; process query terms in **increasing df order** (smallest posting first)
 - For complex Boolean expressions, estimate OR-group sizes and process in ascending size
- **Skip Pointers** – shortcuts inside postings lists to accelerate merging
 - Ideal number of skips $\approx \sqrt{P}$ (where P = posting length)
 - Trade-off: more skips = shorter skip spans but higher storage/comparison cost

Beyond Simple Boolean Retrieval

Enhancement	Description
Term Weights	Incorporate TF, DF, collection statistics (e.g., TF-IDF)
Scoring Functions	Cosine similarity (VSM), probabilistic models (BM25), language models
Ranking	Order results by decreasing relevance scores

Evaluation of Retrieval Systems

Metric	Description
Precision / Recall	Basic set-based measures of relevance
F-measure	Harmonic mean of precision and recall
Precision@K, R-Prec	Incorporate rank cutoff
Average Precision (AP) / Mean Average Precision (MAP)	Average precision across recall levels, then mean across queries
nDCG (Normalized Discounted Cumulative Gain)	Handles graded relevance
MRR (Mean Reciprocal Rank)	Focuses on first relevant result
Kappa (inter-annotator agreement)	Evaluates consistency of relevance judgments
Statistical Testing	Hypothesis testing to compare systems

Experimental Setup (Cranfield Method):

1. **Document Collection** – fixed corpus
2. **Query Set** – predetermined topics/queries
3. **Relevance Judgments** – human-annotated relevance for each query-document pair

All systems under comparison must use the **same collection, same queries, and same relevance judgments** to ensure a fair test.

Additional Topics Covered Later in the Course

- **Web Crawling & PageRank/HITS** – random-surfer model, link analysis
- **Learning-to-Rank** – supervised models that directly optimise ranking metrics
- **Latent Semantic Indexing (LSI)** – dimensionality reduction via SVD
- **Embeddings & Neural IR** – word, sentence, document embeddings; **BERT & LLMs in IR**

Section 2: Storing the Index and Statistical Properties of Text

This section covers index construction algorithms, dictionary and posting-list storage, compression techniques, and statistical regularities of natural language from a 118-page NPTEL presentation ([source](#))

Purpose and Scope

The presentation explains **how an inverted index is built, stored, and compressed** for large information-retrieval systems. It covers:

Area	Topics
Memory hierarchy	RAM vs. secondary storage trade-offs
Index construction algorithms	BSBI, SPIMI
Dictionary representations	Sort-based vs. hash-based
Posting-list representations	Gap encoding, compression
Statistical regularities	Zipf's Law, Heaps' Law
Compression techniques	Dictionary compression, gap encoding

Memory Foundations

Memory Tier	Characteristics	Role in Indexing
Primary memory (RAM)	Very fast, limited capacity, volatile, expensive	Holds in-memory dictionary, posting lists during construction, temporary term-docID pairs
Secondary memory (HDD/SSD)	Slow, virtually unlimited, non-volatile, cheap	Stores final inverted index and intermediate files from external-sorting algorithms

Key point: Because RAM is limited, **large collections cannot be indexed entirely in memory**; external-memory algorithms are required.

Index Construction – Two Phases

Phase	Description
Index Construction (Indexing)	Build posting lists for each term in an incremental, document-by-document fashion
Query Processing (Retrieval)	Use the completed posting lists to answer search queries

The **life-cycle diagram** shows: Document collection → Tokenisation → Term-docID pairs → Dictionary + Postings → Searchable index

Inverted Index Basics

Forward vs. Inverted Index

Index Type	Mapping	Use Case
Forward Index	Document → terms	Document-at-a-time processing
Inverted Index	Term → list of document IDs	Term-at-a-time query processing

Building an Inverted Index

1. **Token Sequencing** – generate (term, docID) pairs for each document
2. **Sorting** – first lexicographically by term, then by docID
3. **Merging** – combine duplicate term-docID pairs, compute **term frequencies (TF)** and **document frequencies (DF)**
4. **Storing** – split into two files:
 - **Dictionary** – sorted list of terms, each with DF, collection frequency (CF), and pointer to postings list
 - **Postings** – ordered list of docIDs (and optionally TF, positions)

Example Dictionary Entry

Term	DF	CF	Posting List (docIDs)
Alfred	8	1	2, 4, 7, 8, 16, 37, 54, 89
Arthur	11	1	1, 2, 5, 9, 11, 25, 31, 53, 68, 91, 94
Batman	4	1	2, 8, 37, 54
Bruce	7	1	2, 4, 8, 11, 18, 37, 54

Storing the Dictionary

Information to Store per Term

Data	Description
Actual term string	Variable length
Document frequency (df)	Integer
Pointer to posting list	Integer/byte offset
(Optional) Term-ID, start offset of postings	Additional metadata

Two Principal Implementations

Method	Description	When to Use
Sort-based dictionary (array, tree)	Terms kept in sorted order → binary search $O(\log n)$. Supports range / prefix queries	Small-to-moderate collections, when range queries are needed
Hash-based dictionary	Terms stored in a hash table → average $O(1)$ lookup. No ordering	Very large or dynamic collections where fast exact-term lookup dominates

Key insight: The choice depends on (i) **number of terms**, (ii) **static vs. dynamic** nature of the vocabulary, and (iii) **need for range queries** (e.g., wildcard search).

Space Considerations

Approach	Space per Term	Notes
Fixed-width entry	28–88 bytes	Term padded to 20–80 bytes, plus 4 bytes df, plus 4 bytes pointer. Wasted space : average English term $\approx$ 8 characters
String-pool alternative	$\approx$ 19 bytes	All terms concatenated in single byte array; 3-byte pointer per term. $\sim 4\times$ reduction vs. fixed-width

Storing Posting Lists

Basic Representation

- For each term, a **sorted list of docIDs** (ascending)
- Example for "Batman": 2 8 37 54

Memory Required per Token (Intermediate Stage)

Item	Size (bytes)
char (raw token)	1–4
term string (max 20–80)	20–80
int / docID	2–4
frequency (tf)	2–4

Total: $\approx$ 28–88 bytes per (term, docID) pair

A modest collection such as **RCV1 ($\approx$ 100M tokens)** would need **2.8–8.8 GB** of RAM just for raw pairs – clearly infeasible for a single machine.

The "Intermediate-Pairs" Problem

- While indexing, documents are parsed one at a time, producing **term-docID pairs**
- The **final postings** for a term are **incomplete** until the entire collection is processed
- Keeping all pairs in memory** for sorting is only possible for very small collections; otherwise **primary memory exhausts**

External Sorting for Large Collections

When primary memory cannot hold all intermediate pairs, **external-memory algorithms** are employed.

Blocked Sort-Based Indexing (BSBI)

Step	Action
1. Divide	Split collection into blocks that fit into RAM
2. Per-block processing	Tokenise $\rightarrow$ generate (term, docID) pairs $\rightarrow$ Sort in-memory by term then docID $\rightarrow$ Write sorted block to disk
3. Merge	Multi-way merge all sorted blocks to produce final inverted index

Assumptions: Dictionary kept in memory; term-to-termID mapping required for compactness. Using raw terms in intermediate files would make them huge and slow to merge.

Single-Pass In-Memory Indexing (SPIMI)

Step	Action
1. Initialise	Empty in-memory dictionary (hash-based)
2. Per-document	Tokenise; for each token, add docID to its postings list (create if absent)
3. Memory limit reached	Write current dictionary ("index block") to disk; clear dictionary; continue
4. Finalisation	Write any remaining dictionary; optionally merge blocks

Key difference: No explicit **sorting** of term-docID pairs; posting lists built directly in dictionary

BSBI vs. SPIMI – Feature Comparison

Feature	BSBI	SPIMI
Indexing Type	Multi-pass, sort-based	Single-pass, in-memory
Dictionary	Sort-based (tree/array)	Hash-based
Sorting Required?	Yes	No
Speed	Slower (sorting overhead)	Faster (single pass)
Memory Usage	Efficient for disk-based systems	Efficient if each block fits; avoids storing all pairs simultaneously
Scalability	Handles arbitrarily large collections	Handles large collections but each block must fit in RAM

 Retrieval Models – From Boolean to Positional

Boolean Retrieval

- Dictionary + postings allow **Boolean queries** (AND, OR, NOT)
- Example: "**Alfred Bruce**" → intersect postings → result {2, 4, 8, 37, 54}
- **Limitation:** Only records **presence/absence**; **no term importance, no frequency, no positional information**

Beyond Boolean – Positional Indexing

Posting list entries can be enriched with:

Data	Purpose
Term-frequency (tf)	Number of occurrences in document
Positions	Exact token offsets within document

Enables:

- **Phrase queries** (exact order, distance = 1)
- **Proximity queries** (ordered/unordered window)
- **Ranking** (TF-IDF, BM25)

 Statistical Properties of Text

Understanding term distribution guides compression and indexing decisions.

Zipf's Law

Empirical law: Term frequency f_t is inversely proportional to its rank r_t

$$f_t = \frac{k}{r_t} \quad \text{or} \quad P(t) = \frac{c}{r_t}$$

Constants:

- N = total number of term occurrences in collection
- c ≈ 0.1 for English
- k = c × N

Implications:

Finding	Impact
Most frequent term ≈ 10% of tokens	Stop-word removal significantly reduces index size
Top 10 terms ≈ 30% of all tokens	Common terms dominate storage
Top 20 terms ≈ 36%; Top 50 ≈ 45%	Long inverted lists belong to very frequent terms
Ignoring/down-weighting stop-words	Dramatically reduces index size, though may be needed for proximity queries

Visualization: Log-log plot of frequency vs. rank yields straight line with slope -1

Heaps' Law

Describes **vocabulary growth** with corpus size

$$v = k \cdot n^\beta$$

Where:

- v = number of **unique** words (vocabulary size)
- n = total number of word occurrences (corpus size)
- k ≈ 10–100 (collection-dependent)
- β ≈ 0.5 (typically)

Implications:

- **Rapid early growth** of new terms, then **sub-linear** (slower) growth
- Even huge corpora continually produce **new terms** (proper nouns, misspellings, invented words)
- Indexes must be designed to accommodate vocabulary growth

Section 2: Storing the Index – Compression, Construction, and Statistical Properties

This section covers index storage mechanisms, construction algorithms for large-scale collections, and statistical properties of text that inform compression decisions from a 118-page NPTEL presentation ([source](#))

Note: This section builds upon the foundational concepts introduced in Section 1, with deeper focus on **engineering constraints**, **external-memory algorithms**, and **compression techniques** for production-scale IR systems.

Memory Hierarchy and Constraints

Memory Tier	Speed	Capacity	Volatility	Cost	IR Usage
Primary (RAM)	Very fast (ns)	Limited (GBs)	Volatile	Expensive	In-memory dictionary, temporary posting lists, query processing

Secondary (HDD/SSD)	Slow (ms/μs)	Virtually unlimited	Non-volatile	Cheap	Final inverted index, intermediate sorted blocks
---------------------	--------------	---------------------	--------------	-------	--

Critical constraint: Large collections cannot be indexed entirely in memory; **external-memory algorithms** are essential for scalability.

 Index Construction Algorithms

Two-Phase Lifecycle

Phase	Activity	Output
Construction (Indexing)	Build posting lists incrementally, document-by-document	Incomplete posting lists, intermediate files
Query Processing (Retrieval)	Use completed posting lists to answer queries	Ranked results

The Intermediate-Pairs Problem

During indexing:

- Documents parsed **one at a time** → produce **(term, docID) pairs**
- Final postings for a term **incomplete** until entire collection processed
- Memory bottleneck:** RCV1 (100M tokens) needs **2.8–8.8 GB** just for raw pairs

Solution: External sorting algorithms that work within RAM constraints

 Blocked Sort-Based Indexing (BSBI)

Step	Action	Details
1. Divide	Split collection into blocks fitting in RAM	Size determined by available memory
2. Per-block processing	Tokenise → generate pairs → sort in-memory by term then docID → write sorted block to disk	Creates intermediate index files
3. Merge	Multi-way merge all sorted blocks	Produces final inverted index

Assumptions: Dictionary kept in memory; term-to-termID mapping for compactness

 Single-Pass In-Memory Indexing (SPIMI)

Step	Action	Key Difference from BSBI
1. Initialise	Empty hash-based dictionary in memory	Hash vs. sort-based
2. Per-document	Tokenise; add docID to postings list directly	No sorting of pairs
3. Memory limit	Write dictionary ("index block") to disk; clear; continue	Dynamic block creation
4. Finalisation	Write remaining dictionary; optionally merge blocks	Flexible merging

BSBI vs. SPIMI Comparison

Feature	BSBI	SPIMI
Indexing type	Multi-pass, sort-based	Single-pass, in-memory
Dictionary structure	Sort-based (tree/array)	Hash-based
Sorting required	✓ Yes	✗ No
Speed	Slower (sorting overhead)	Faster (single pass)
Memory efficiency	Efficient for disk-based; blocks must fit	Efficient if each block fits; avoids storing all pairs
Scalability	Arbitrarily large (external merge)	Large collections; each block must fit RAM

Dictionary Storage Implementations

Sort-Based vs. Hash-Based

Aspect	Sort-Based (Array/Tree)	Hash-Based
Lookup time	O(log n) binary search	O(1) average
Range queries	✓ Supported	✗ Not supported
Prefix queries	✓ Supported	✗ Not supported
Dynamic updates	Slower (rebalancing)	Faster
Best for	Static vocabularies, wildcard search	Large dynamic collections

Space Optimization: String Pool

Approach	Bytes per Term	Calculation
Fixed-width entry	28–88	20–80 (padded term) + 4 (df) + 4 (pointer)
String-pool	≈ 19	4 (df) + 4 (posting-pointer) + 3 (string-pointer) + ~8 (avg term)

~4× space reduction with string-pool approach

Statistical Properties of Text

Zipf's Law

Empirical law: Term frequency is inversely proportional to its rank

$$f_t = \frac{k}{r_t} \quad \text{or} \quad P(t) = \frac{c}{r_t}$$

Constant	Value	Meaning
N	Collection-specific	Total term occurrences
c	≈ 0.1 (English)	Proportionality constant
k	c × N	Scaled constant

Key Implications:

Rank	Approximate % of Tokens	Example Terms
1st	~10%	"the"
2nd	~5%	"of"
3rd	~3%	"and"
Top 10	~30%	Common stop-words
Top 20	~36%	Very frequent terms
Top 50	~45%	High-frequency vocabulary

Practical impact: Long inverted lists belong to very frequent terms → dominate index size. Ignoring or down-weighting stop-words dramatically reduces index size.

Heaps' Law

Describes **vocabulary growth** with corpus size

$$v = k \cdot n^{\beta}$$

Variable	Meaning	Typical Value
v	Unique words (vocabulary size)	Collection-dependent
n	Total word occurrences (corpus size)	Measured in tokens
k	Scaling constant	10–100 (collection-dependent)
β	Growth exponent	≈ 0.5

Implications:

- **Rapid early growth** of new terms, then **sub-linear** (slower) growth
- Even huge corpora continually produce **new terms** (proper nouns, misspellings, invented words)
- Indexes must be designed to accommodate **ongoing vocabulary growth**

Section 3: Getting Started with PyLucene – Indexing

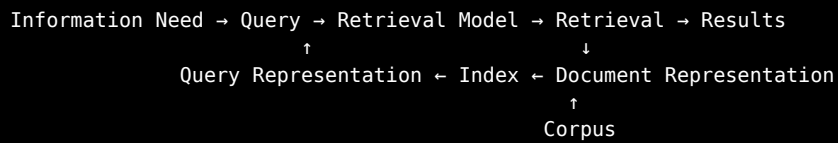
This section provides practical guidance on using PyLucene for building search indexes, including installation, document modeling, field types, and complete indexing workflows from a 78-page NPTEL presentation ([source](#))

Context and Objectives

Aspect	Details
Course/Module	Introduction to Information Retrieval – practical hands-on with PyLucene
What is PyLucene	Python wrapper for Apache Lucene
Primary Goals	1. Explain IR fundamentals 2. Survey IR research platforms 3. Introduce Apache Lucene architecture 4. Demonstrate PyLucene indexing from Python 5. Detail complete indexing workflow 6. Provide concrete code examples

Information Retrieval Foundations

Graphical Representation of IR



- **Document representation** → **Index** → **Query representation**
- **Corpus** = collection of documents

Popular IR Research Systems (Chronological)

System	Developed By	Initial Release	Language
SMART	Cornell University	1960–1970s	C
Indri	UMass Amherst & CMU (Lemur Project)	2002	C++
Galago	UMass Amherst (Lemur Project)	2011	Java
Terrier	University of Glasgow	2004	Java
Lucene	Doug Cutting (Apache Software Foundation)	1999–2000	Java

Lucene is highlighted as **non-academic** but valued for: **Robustness, Flexibility, Efficiency with big data**

Apache Lucene Overview

Core Facts

Aspect	Details
Language	Java-based IR engine
Ecosystem	Apache Open-Source
Primary use	Full-text search
Downstream projects	ElasticSearch, Solr, Tika, Nutch

Strengths

- Simple, fast API
- Supports **concurrent indexing & searching**
- Implements variety of **retrieval models** (BM25, TF-IDF, language models, etc.)

Limitations

Limitation	Mitigation
No built-in parsers for binary office formats (.doc, .pdf)	Pair with external parsers (Apache Tika, PDFBox)
"Can't be used out-of-the-box" for rich document formats	Pre-processing pipeline required

PyLucene – Python Wrapper for Lucene

What is PyLucene?

Aspect	Details
Definition	Python extension exposing Java Lucene classes to Python
Goal	Allow Python developers to use Lucene's indexing & searching capabilities
Version used	PyLucene 10 (compatible with Lucene 10.x)

Architecture

```
Python Code → JCC (C++ code generator) → JNI → Embedded Java VM → Lucene Java Classes
```

Component	Function
Embedded Java VM	JVM runs inside Python process
JCC	Creates C++ glue code enabling Python → Java calls
JNI	Java Native Interface for cross-language calls
Sources for JCC	Bundled with PyLucene source distribution

 Installing PyLucene

Prerequisites

Component	Minimum Version
Python	3.x
Java	≥ 21
Build tools	Ant and Ivy
PyLucene source	https://lucene.apache.org/pylucene/
JCC	Must be installed before building PyLucene

Build & Installation Steps

```
# 1. Edit Makefile → set JAVA_HOME, JCC path, etc.
make           # may require sudo
make install   # long (≈30–45 min)
make test      # verify success
```

"Easy" Installation via Docker

Aspect	Details
Problem	Manual installation error-prone due to Java dependencies
Solution	Pre-configured Docker image (coady/pylucene on Docker Hub)
Benefits	Isolated environment, all dependencies pre-installed, VS Code remote container support

Lucene Document Model

Core Concepts

Concept	Definition
Document	Atomic unit of indexing/search (web page, PDF, email, DB row)
Fields	Name/value pairs inside a Document
Flexibility	Different documents may have different field sets

Field Types & Options

Option	Meaning	Use Case
Indexed	Contributes to inverted index → searchable	All searchable content
Non-indexed	Ignored by search engine (only stored)	Display-only metadata
Analyzed (Tokenized)	Processed by Analyzer (tokenisation, stemming, stop-words)	Full-text fields
Non-analyzed	Treated as single token	IDs, URLs, dates
Stored	Value kept in index for retrieval	Display in UI
Term Vectors	Stores positional/post-offset data	Highlighting, similarity, query-expansion

FieldType (Low-Level Configuration)

Property	Controls
stored	Whether original value retained
indexed	Whether field in inverted index
tokenized	Whether value broken into tokens
storeTermVectors	Whether term vectors kept
indexOptions	Granularity: DOCS, DOCS_AND_FREQS, DOCS_AND_FREQS_AND_POSITIONS, etc.
storeTermVectorPositions/Offsets	Positional/offset data in term vectors

IndexOptions Enum (Lucene 10):

Option	Stored Data
DOCS	Only document IDs
DOCS_AND_FREQS	Add term frequencies
DOCS_AND_FREQS_AND_POSITIONS	Add positions (phrase/Span queries)
DOCS_AND_FREQS_AND_POSITIONS_AND_OFFSETS	Add character offsets
NONE	Field not indexed

Important distinction: `setIndexOptions(DOCS_AND_FREQS_AND_POSITIONS)` stores positions in the **main inverted index** for searching; `setStoreTermVectorPositions(True)` stores positions in **term vectors** for post-processing (highlighting, custom analysis).

 Pre-defined Field Sub-classes (Lucene 10)

Class	Indexed	Tokenized	Typical Use
TextField	✔ Yes	✔ Yes (analyzed)	Full-text content (title, body, abstract)
StringField	✔ Yes	✗ No (single term)	Exact matches (IDs, tags, categories)
Point Fields (IntField, LongField, FloatField, DoubleField)	✔ Yes (numeric)	N/A	Numeric ranges, sorting, spatial queries
StoredField	✗ No	✗ No	Metadata for display only (URL, image link)

Quick Reference Syntax (Java-style, usable via PyLucene):

```
// TextField
new TextField("title", "The Prestige", Field.Store.YES)

// StringField
new StringField("origin", "Indian", Field.Store.YES)

// IntField (numeric)
new IntField("year", 2008, Field.Store.YES)

// StoredField (metadata only)
new StoredField("url", "https://example.com/", Field.Store.YES)
```

Comparison Table (Lucene 10):

Field	Indexed	Tokenized	Stored	Term Vectors
TextField	Yes	Yes	Optional	Tunable
StringField	Yes	No	Optional	Tunable
Point Fields	Yes (numeric)	N/A	Optional	Tunable
StoredField	No	No	Yes	Tunable

Section 3: PyLucene Indexing – Practical Implementation and Document Construction

This section provides hands-on guidance for building search indexes with PyLucene, including complete indexing workflows, field design, and analyzer configuration from a 78-page NPTEL presentation ([source](#))

 Building Documents in PyLucene

Example Document: Movie "The Dark Knight"

Field	Example Value	Field Type
title	"The Dark Knight"	TextField
director	"Christopher Nolan"	StringField
plot	(long description)	TextField (unstored)

release_year	2008	IntField
origin	"American"	StringField
cast	"Christian Bale, Heath Ledger, ..."	TextField
genre	"superhero"	StringField
wiki_page	URL to Wikipedia article	StoredField
image_url	URL to poster image	StoredField (only)

Document Construction (Pseudo-Python)

```
from org.apache.lucene.document import Document, TextField, StringField, IntField, StoredField, Field

doc = Document()
doc.add(TextField("title", title, Field.Store.YES))
doc.add(StringField("director", director, Field.Store.YES))
doc.add(TextField("plot", plot, Field.Store.NO)) # indexed but not stored
doc.add(IntField("year", int(year), Field.Store.YES))
doc.add(StoredField("url", wiki_url)) # stored but not indexed
# ... add remaining fields as needed
writer.addDocument(doc)
```

Dataset: Wikipedia Movie Plot

Attribute	Details
Source	Kaggle
Fields	Release Year, Title, Origin/Ethnicity, Director, Cast, Genre, Wiki Page, Plot
Goal	Index each row as Lucene Document, enable IR queries

Example queries:

- director:nolan
- plot:batman

 Indexing Pipeline in PyLucene

High-Level Flow

```
[Raw Collection] → [Document Parser] → [Analyzer] → [IndexWriter] → [Directory (RAM/FS/...)]
```

Component	Function
Document Parser	Convert source format (CSV, TSV, JSON, HTML, PDF, WARC, TREC) to field values
Analyzer	Language-specific tokenisation, stop-word removal, stemming
IndexWriter	Writes Documents to a Directory (e.g., FSDirectory on disk)

Setting Up Writer & Analyzer

```
import lucene
from java.io import File
from org.apache.lucene.analysis.standard import StandardAnalyzer
from org.apache.lucene.store import FSDirectory
from org.apache.lucene.index import IndexWriter, IndexWriterConfig

lucene.initVM() # start embedded JVM
index_path = File("index/").toPath()
directory = FSDirectory.open(index_path) # could also use RAMDirectory, etc.
config = IndexWriterConfig(StandardAnalyzer())
writer = IndexWriter(directory, config)
```

Parsing CSV with Pandas

```
import pandas as pd

def read_csv_file(path):
    df = pd.read_csv(path)
    for i, row in df.iterrows():
        print("Doc {} – Title: {}".format(i, row['Title']))
        # Build Lucene Document here
```

Note: Lucene maintains its own internal doc-ids; the printed docid is for human reference only.



Tokenisation Demonstrations

Standard Analyzer

```
from java.io import StringReader
from org.apache.lucene.analysis.standard import StandardAnalyzer
from org.apache.lucene.analysis.tokenattributes import CharTermAttribute

text = "This is how we do it."
analyzer = StandardAnalyzer()
stream = analyzer.tokenStream("", StringReader(text))
stream.reset()
tokens = []
while stream.incrementToken():
    tokens.append(stream.getAttribute(CharTermAttribute.class).toString())
print(tokens) # ['this', 'is', 'how', 'we', 'do', 'it']
```

English Analyzer (Stemming, Stop-word Removal)

```
from org.apache.lucene.analysis.en import EnglishAnalyzer
analyzer = EnglishAnalyzer()
# same token-stream loop → ['quick', 'brown', 'fox', 'jump', 'over', 'lazi', 'dog']
```

Note: "lazy" → "lazi" (Porter stemming), "jumps" → "jump"

Section 4: Non-English Text Analysis and Multilingual IR with PyLucene

This section covers multilingual text processing, language-specific analyzers, index inspection with Luke, and practical exercises for Indian language indexing from a 40-page NPTEL presentation ([source](#))

PyLucene Analyzers and Language Support

Analyzer Basics

Analyzer in Lucene: Tokenises text, removes stop-words, and optionally stems tokens

Critical principle: The choice of analyzer **must match the language** of the text being indexed.

Typical Python import pattern:

```
# Generic pattern
# from org.apache.lucene.analysis.ln import LanguageAnalyzer
# analyzer = LanguageAnalyzer()

# German example
from org.apache.lucene.analysis.de import GermanAnalyzer
analyzer = GermanAnalyzer()

# English example
from org.apache.lucene.analysis.en import EnglishAnalyzer
analyzer = EnglishAnalyzer()
```

Key point: Set the analyzer according to the language of each field; using a mismatched analyzer degrades tokenisation, stop-word removal, and stemming.

Complete Language Analyzer Reference

Code	Language	Code	Language	Code	Language
ar	Arabic	et	Estonian	lv	Latvian
bg	Bulgarian	eu	Basque	ne	Nepali
bn	Bengali	fa	Persian	nl	Dutch
br	Breton	fi	Finnish	no	Norwegian
ca	Catalan	fr	French	pt	Portuguese
cjk	Chinese-Japanese-Korean	ga	Irish	ro	Romanian
ckb	Sorani	gi	Galician	ru	Russian
cz	Czech	hi	Hindi	sr	Serbian
da	Danish	hu	Hungarian	sv	Swedish
de	German	hy	Armenian	ta	Tamil
el	Greek	id	Indonesian	te	Telugu
en	English	it	Italian	th	Thai
es	Spanish	lt	Lithuanian	tr	Turkish

Viewing and Inspecting Indexes with Luke

What is Luke?

Luke (Lucene Index Toolbox): GUI utility for browsing Lucene indexes, inspecting fields, term statistics, postings, stored fields

Critical constraint: Version-specific – Luke version must match Lucene index version (e.g., Luke 10 only works with Lucene 10.x)

Obtaining and Running Luke

Method	Steps	Complexity
Build from source (Windows 11)	1. Install JDK 21.08 2. Install Strawberry Perl 5.32.1.1 3. Install Gradle 8.10 4. Download Lucene 10.0.0 source 5. Run gradle or gradlew assemble	High
Build from binary	1. Install JDK 21.08 2. Download compiled Lucene 10.0.0 binary 3. Extract, navigate to bin/, launch luke.bat	Medium
Pre-compiled download	Direct: https://tinyurl.com/get-luke-10 or https://luke-10	Low

Warning: "Luke is very much version-dependent!"



Practical PyLucene Exercises

#	Task	Learning Outcome
1	Index same collection with different settings (analyzers, storage options); compare index size in Luke	Observe analyzer/storage impact on footprint
2	Print stored values in index; contrast Stored vs. Unstored fields	Understand stored flag impact on retrieval
3	Print stored values of all documents	Practice IndexReader/IndexSearcher iteration
4	Apply different analyzers to different fields of single document	Learn PerFieldAnalyzerWrapper for multilingual indexing
5	Index Indian-language text collection (Bengali, Tamil, Telugu, Hindi)	Experience language-specific tokenisation, Unicode handling
6	Index text from .txt file	Simple file-to-index pipeline



Reflection Questions – Deepening Understanding

Field & Storage

Question	Answer
StringField vs. TextField	StringField: untokenised , exact matches (IDs, URLs). TextField: tokenised , indexed, full-text search

When to store?	Store when retrieving original text needed (titles, snippets). Index-only saves space if only scoring required
Effect on Luke	Stored fields appear in "Stored Fields" tab; unstored fields absent but contribute to term statistics

Analyzers

Aspect	Impact
Tokenisation, stop-words, stemming	Language-specific pipelines (e.g., EnglishAnalyzer: Porter stemming, English stop-words)
StandardAnalyzer vs. WhitespaceAnalyzer vs. EnglishAnalyzer	<i>Standard</i> : Unicode-aware, punctuation removal, stop-words. <i>Whitespace</i> : Splits on whitespace only. <i>English</i> : Adds stemming to Standard
Single analyzer for multiple languages	Risk : Incorrect tokenisation (German umlauts, Hindi Devanagari dropped)
PerFieldAnalyzerWrapper	Field-level analyzer assignment for multilingual indexing (e.g., title→English, body_de→German)

Index Inspection

Concept	Explanation
numDocs() vs. maxDoc()	numDocs(): live (non-deleted) documents. maxDoc(): includes deleted docs (total slots)
Token stream in Luke	Shows analyzed token sequence ; comparing with original reveals analyzer effects

Document Design

Consideration	Guidance
Long content vs. short title	Longer fields generate more term statistics, potentially dominating scoring. Short titles can be boosted
Multiple fields vs. single concatenated field	Separate fields enable field-level weighting , field- restricted queries, per-field tokenisation control

Stop-words & Term Importance

Aspect	Rationale
Why stop-words?	High-frequency, low-information terms increase index size and slow retrieval without improving relevance
Rare but important terms	Keep unstopped ; optionally boost during indexing or query time (e.g., "IISER")

Advanced Multilingual Indexing

Strategy	Implementation
Organising analyzers & fields	Create per-language sub-fields (content_en, content_de, content_fr); use PerFieldAnalyzerWrapper
Impact on ranking	Proper language-specific tokenisation yields more accurate term statistics → better BM25/LM scores

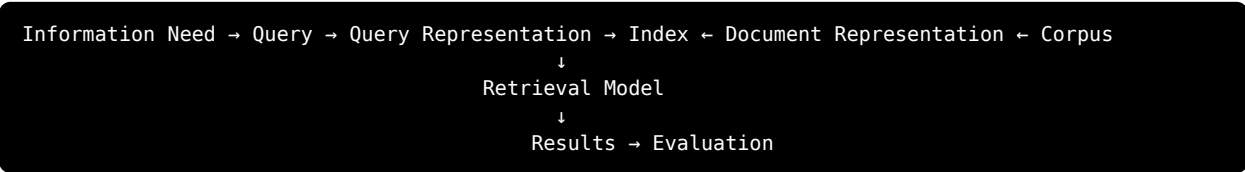
Performance considerations	More fields increase index size and query complexity; balance coverage vs. overhead
----------------------------	--

 Indian Language Text Collections (Data Sources)

Language	Kaggle Dataset URL	Notes
Bengali	https://www.kaggle.com/datasets/mdismw/indian-language-text-datasets-of-various-topics	Various topics, useful for multilingual experiments
Tamil	https://www.kaggle.com/datasets/sudalairajkumar/tamil-nlp	Tokenised corpora and sentiment data
Telugu	https://www.kaggle.com/datasets/chintanbhatnagar/telugu-corpus	Large and diverse, suitable for language modeling
Hindi	https://www.kaggle.com/datasets/disibig/hindi-text-short-and-large-summarization-corpus	Short & long texts, includes summarisation data

 Suggested for Practice Problem 5: Index Indian language text collection

 Fundamentals of IR – Graphical Model



Component	Purpose
Document representation	Decides what data (terms, frequencies, positions) stored in index
Index	Data structure enabling fast term → document lookup
Query representation	How user's need transformed into searchable form
Retrieval model	Algorithm deciding relevance (Boolean, Vector Space, Language Model)
Evaluation	Measures: precision, recall, MAP, nDCG

 Document & Index Representations

Binary Full-Text (Boolean Retrieval)

Characteristic	Description
Representation	Bag-of-words (no frequencies, no positions)
Structure	Binary incidence matrix (1 = presence, 0 = absence)
Operations	AND, OR, NOT
Result	Unranked Boolean retrieval

Variable-Length Inverted Lists with Term-Frequencies

Component	Format
Dictionary	Term → pointer to postings
Posting list	List of (docID, tf) pairs
Limitation	No positional data → cannot support proximity operators

Positional (Full-Text) Index

Enhancement	Format
Posting format	docID, tf [pos1, pos2, ..., pos_tf]
Enables	Phrase queries (exact order, distance = 1)
	Ordered/Unordered window queries (proximity)
Example	information OW/2 science

Query Operators Beyond Simple Boolean

Proximity Operators

Operator	Syntax	Meaning	Example
Ordered window (OW/N)	A OW/N B	A appears no more than N terms before B (order preserved)	(information OW/2 science) matches "information retrieval science"

Section 4: Non-English Text Analysis and Multilingual IR with PyLucene

This section covers multilingual text processing, language-specific analyzers for global languages including Indian languages, and practical exercises for building cross-lingual search systems from a 40-page NPTEL presentation ([source](#))

Note: This section synthesizes practical implementation guidance for multilingual IR, with extensive coverage of **Indian language support** and **field-level analyzer configuration**.

PyLucene Analyzers and Language Support

Analyzer Fundamentals

Analyzer in Lucene: Component that **tokenises text**, **removes stop-words**, and **optionally stems tokens**

Critical Principle:

The choice of analyzer **must match the language** of the text being indexed. Using a mismatched analyzer degrades tokenisation, stop-word removal, and stemming.

Standard Import Pattern:

```
# German
from org.apache.lucene.analysis.de import GermanAnalyzer
analyzer = GermanAnalyzer()

# English
from org.apache.lucene.analysis.en import EnglishAnalyzer
```



```
analyzer = EnglishAnalyzer()

# French
from org.apache.lucene.analysis.fr import FrenchAnalyzer
analyzer = FrenchAnalyzer()
```

Key point: Set the analyzer according to the language of each field

Complete Language Analyzer Reference

Code	Language	Code	Language	Code	Language
ar	Arabic	et	Estonian	lv	Latvian
bg	Bulgarian	eu	Basque	ne	Nepali
bn	Bengali	fa	Persian	nl	Dutch
br	Breton	fi	Finnish	no	Norwegian
ca	Catalan	fr	French	pt	Portuguese
cjk	Chinese-Japanese-Korean	ga	Irish	ro	Romanian
ckb	Sorani	gi	Galician	ru	Russian
cz	Czech	hi	Hindi	sr	Serbian
da	Danish	hu	Hungarian	sv	Swedish
de	German	hy	Armenian	ta	Tamil
el	Greek	id	Indonesian	te	Telugu
en	English	it	Italian	th	Thai
es	Spanish	lt	Lithuanian	tr	Turkish

Indian languages supported: Bengali (bn), Hindi (hi), Tamil (ta), Telugu (te), plus Nepali (ne)

Viewing and Inspecting Indexes with Luke

Luke Overview

Luke (Lucene Index Toolbox): GUI utility for browsing Lucene indexes, inspecting fields, term statistics, postings, stored fields

Critical Constraint:

Version-specific – Luke version must match Lucene index version (e.g., Luke 10 works only with Lucene 10.x)

Installation Methods

Method	Complexity	Steps
Build from source (Windows 11)	High	1. Install JDK 21.08 2. Install Strawberry Perl 5.32.1.1 3. Install Gradle 8.10 4. Download Lucene 10.0.0 source 5. Run gradle or gradlew assemble

Build from binary	Medium	1. Install JDK 21.08 2. Download compiled Lucene 10.0.0 binary 3. Extract, navigate to bin/, launch luke.bat
Pre-compiled download	Low	Direct: https://tinyurl.com/get-luke-10 or https://luke-10

Warning repeated: "Luke is very much version-dependent!"

Practical PyLucene Exercises

#	Task	Learning Outcome
1	Index same collection with different settings (analyzers, storage options); compare index size in Luke	Observe analyzer/storage impact on footprint
2	Print stored values in index; contrast Stored vs. Unstored fields	Understand stored flag impact on retrieval
3	Print stored values of all documents	Practice IndexReader/IndexSearcher iteration
4	Apply different analyzers to different fields of single document	Learn PerFieldAnalyzerWrapper for multilingual indexing
5	Index Indian-language text collection (Bengali, Tamil, Telugu, Hindi)	Experience language-specific tokenisation, Unicode handling
6	Index text from .txt file	Simple file-to-index pipeline

Reflection Questions – Deepening Understanding

Field and Storage Design

Question	Answer
StringField vs. TextField	StringField: untokenised , exact matches (IDs, URLs). TextField: tokenised , indexed, full-text search
When to store?	Store when original text retrieval needed (titles, snippets). Index-only saves space if only scoring required
Effect on Luke	Stored fields appear in "Stored Fields" tab; unstored fields absent but contribute to term statistics

Analyzer Selection and Impact

Aspect	Guidance
Tokenisation, stop-words, stemming	Language-specific pipelines (e.g., EnglishAnalyzer: Porter stemming, English stop-words)
StandardAnalyzer vs. WhitespaceAnalyzer vs. EnglishAnalyzer	<i>Standard</i> : Unicode-aware, punctuation removal, stop-words. <i>Whitespace</i> : Splits on whitespace only. <i>English</i> : Adds stemming to Standard
Single analyzer for multiple languages	Risk : Incorrect tokenisation (German umlauts, Hindi Devanagari dropped)

PerFieldAnalyzerWrapper	Field-level analyzer assignment for multilingual indexing (e.g., title→English, body_de→German)
--------------------------------	--

Index Inspection Techniques

Concept	Explanation
numDocs() vs. maxDoc()	numDocs(): live (non-deleted) documents. maxDoc(): includes deleted docs (total slots)
Token stream in Luke	Shows analyzed token sequence ; comparing with original reveals analyzer effects

Document Design Strategies

Consideration	Best Practice
Long content vs. short title	Longer fields generate more term statistics, potentially dominating scoring. Short titles can be boosted
Multiple fields vs. single concatenated field	Separate fields enable field-level weighting , field-restricted queries, per-field tokenisation control

Stop-words and Term Importance

Aspect	Rationale
Why stop-words?	High-frequency, low-information terms increase index size and slow retrieval without improving relevance
Rare but important terms	Keep unstopped ; optionally boost during indexing or query time (e.g., "IISER")

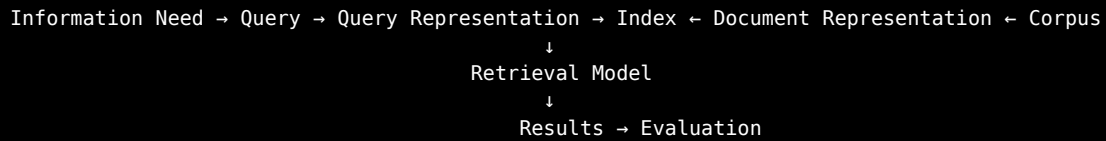
Advanced Multilingual Indexing

Strategy	Implementation
Organising analyzers & fields	Create per-language sub-fields (content_en, content_de, content_fr); use PerFieldAnalyzerWrapper
Impact on ranking	Proper language-specific tokenisation → more accurate term statistics → better BM25/LM scores
Performance considerations	More fields increase index size and query complexity; balance coverage vs. overhead

 Indian Language Text Collections

Language	Kaggle Dataset	Characteristics
Bengali	https://www.kaggle.com/datasets/mdismwaha/text-datasets-of-various-topics	Various topics, useful for multilingual experiments
Tamil	https://www.kaggle.com/datasets/sudalairajkumar/tamil-nlp	Tokenised corpora and sentiment data
Telugu	https://www.kaggle.com/datasets/chinthaarav/corpus	Large news corpus, suitable for language modeling
Hindi	https://www.kaggle.com/datasets/disibig/hindi-text-short-and-large-summarization-corpus	Short & long texts, includes summarisation data

IR Fundamentals – Graphical Model



Component	Function
Document representation	Determines stored data (terms, frequencies, positions)
Index	Fast term → document lookup structure
Query representation	Transforms user need to searchable form
Retrieval model	Relevance algorithm (Boolean, VSM, Language Model)
Evaluation	Metrics: precision, recall, MAP, nDCG

Document and Index Representations

Binary Full-Text (Boolean Retrieval)

Feature	Description
Representation	Bag-of-words, no frequencies or positions
Structure	Binary incidence matrix (1 = presence, 0 = absence)
Operations	AND, OR, NOT
Result	Unranked Boolean retrieval
Limitation	No ranking capability

Variable-Length Inverted Lists with Term-Frequencies

Component	Format
Dictionary	Term → pointer to postings
Posting list	(docID, tf) pairs
Limitation	No positional data → no proximity operators

Positional (Full-Text) Index

Enhancement	Format	Capability
Posting format	docID, tf [pos1, pos2, ..., pos_tf]	Exact position tracking
Enables	Phrase queries (exact order, distance=1)	"information retrieval"
	Proximity queries (ordered/unordered window)	information OW/2 science

Query Operators Beyond Simple Boolean

Proximity Operators

Operator	Syntax	Meaning	Example Match
Ordered window (OW/N)	A OW/N B	A appears $\leq N$ terms before B, order preserved	(information OW/2 science) matches "information retrieval science"
Unordered window (UW/N)	A UW/N B	A and B within N terms, any order	(information UW/2 science) matches "science of information"

Putting it All Together

Core IR Concepts

Information Retrieval is the process of locating unstructured textual documents that satisfy a user's information need.

- **Document representation** follows a two-step pipeline:
 1. **Text processing** – tokenisation, case-folding, stop-word removal, stemming/lemmatization.
 2. **Indexing** – building an inverted index (dictionary + posting lists).
- **Retrieval models** evolve from simple Boolean logic to ranked approaches such as TF-IDF, BM25, language models, and neural embeddings (BERT/LLMs).
- **Evaluation** relies on set-based measures (precision, recall) and rank-aware metrics (MAP, nDCG, MRR) applied to a common Cranfield-style test collection.

Inverted Index Construction & Storage

Aspect	Key Idea	Practical Choice
Dictionary	Maps each term to its posting-list pointer and document frequency.	Sorted array for range queries or hash table for $O(1)$ lookups.
Posting list	Sorted docIDs (optionally with tf, positions, offsets).	Store gaps and apply compression (e.g., variable-byte, gamma).
Memory hierarchy	RAM holds temporary term-docID pairs; disk stores the final index.	Use BSBI (blocked sort) or SPIMI (single-pass) depending on RAM availability.
Statistical regularities	Zipf's law → a few terms dominate postings; Heaps' law → vocabulary grows sub-linearly.	Prioritise stop-word handling and efficient gap encoding for frequent terms.

Skip pointers are placed roughly at $\sqrt{P}$ intervals (P = posting length) to accelerate Boolean merging.

PyLucene Workflow

1. **Initialize the JVM** (`lucene.initVM()`).
2. **Create a StandardAnalyzer (or language-specific analyzer)** to tokenise text.
3. **Configure an IndexWriter** with an `FSDirectory` (or `RAMDirectory`).
4. **Build Document objects** using appropriate field types:
 - `TextField` – indexed + analyzed, optionally stored.
 - `StringField` – indexed + untokenised (ideal for IDs, URLs).
 - `IntField` / `FloatField` – numeric indexing for range queries.

- `StoredField` – kept for display only.
5. **Add documents** via `writer.addDocument(doc)` and **commit** when finished.
 6. **Search** with an `IndexSearcher` and a matching analyzer.

Sample Field Construction (Python-style)

```
doc.add(TextField("title", title, Field.Store.YES))
doc.add(StringField("origin", origin, Field.Store.YES))
doc.add(IntField("year", int(year), Field.Store.YES))
doc.add(StoredField("url", wiki_url))
```

Multilingual & Non-English Analysis

- **Analyzers** are language-specific (e.g., `GermanAnalyzer`, `EnglishAnalyzer`, `TamilAnalyzer`).
- For mixed-language collections, use a **PerFieldAnalyzerWrapper** to assign each field its proper analyzer.
- Indian-language corpora (Bengali, Hindi, Tamil, Telugu) benefit from Unicode-aware tokenisers and language-appropriate stop-word lists.

Language	Analyzer Class
Arabic	<code>ArabicAnalyzer</code>
Bengali	<code>BengaliAnalyzer</code>
Hindi	<code>HindiAnalyzer</code>
Tamil	<code>TamilAnalyzer</code>
...	(full list includes 30+ languages)

Index Exploration with Luke

- **Luke** visualises the dictionary, term statistics, stored fields, and postings.
- Compatibility: the Luke version must match the Lucene index version (e.g., Luke 10 ↔ Lucene 10).
- Useful checks:
 1. **`numDocs()` vs. `maxDoc()`** – live vs. total slots.
 2. Token stream displayed for a field reveals the effect of the chosen analyzer (stop-word removal, stemming).

Evaluation & Benchmarking

- Adopt a **fixed collection, fixed query set, fixed relevance judgments** to ensure fair comparison.
- Apply statistical tests (t-test, paired t-test) when comparing metric scores across systems.

Connecting the Pieces

- **Text processing** determines the quality of the inverted index; the index's structure (dictionary + compressed postings) dictates retrieval speed.
- **Analyzers** bridge language characteristics with tokenisation, directly influencing term statistics that feed ranking models.
- **PyLucene** provides a Pythonic bridge to Lucene's robust indexing/search engine, enabling rapid prototyping of multilingual IR pipelines.
- **Evaluation metrics** close the loop, turning raw relevance scores into actionable insights for system improvement.